

By 27.02.25

**This file is the compilation from
press.bmstu.ru**

The book's formatting could probably be corrupted

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»

А.В. Брагин, И.П. Иванов

Операционные системы

Учебно-методическое пособие


Москва
ИЗДАТЕЛЬСТВО
МГТУ им. Н.Э. Баумана
2024

УДК 681.3.066
ББК 32.972.11
Б87

Издание доступно в электронном виде по адресу
<https://press.bmstu.ru/catalog/item/8226/>

Факультет «Информатика и системы управления»
Кафедра «Теоретическая информатика и компьютерные технологии»

*Рекомендовано Научно-методическим советом
МГТУ им. Н.Э. Баумана в качестве учебно-методического пособия*

Рецензент

канд. техн. наук, доцент *И.И. Троицкий*

Брагин, А. В.

Б87 Операционные системы : учебно-методическое пособие /
А. В. Брагин, И. П. Иванов. — Москва : Издательство МГТУ
им. Н. Э. Баумана, 2024. — 33, [3] с. : ил.

ISBN 978-5-7038-6293-3

Приведено описание четырех лабораторных работ по курсу «Операционные системы», посвященных ознакомлению с системами сборки двух типов современных операционных систем, модификации ядер операционных систем, а также программированию в режиме ядра.

Для студентов 2-го курса, обучающихся по направлению подготовки 01.03.02 «Прикладная математика и информатика». Пособие также может быть полезным студентам, обучающимся по направлению подготовки 09.03.01 «Информатика и вычислительная техника», а также аспирантам научной специальности 2.3.2 «Вычислительные системы и их элементы» (кафедра «Теоретическая информатика и компьютерные технологии» МГТУ им. Н.Э. Баумана).

УДК 681.3.066
ББК 32.972.11



*Уважаемые читатели! Пожелания, предложения, а также сообщения о замеченных опечатках и неточностях Издательство просит направлять по электронной почте:
press@bmstu.ru*

ISBN 978-5-7038-6293-3

© МГТУ им. Н.Э. Баумана, 2024
© Оформление. Издательство
МГТУ им. Н.Э. Баумана, 2024

Предисловие

Настоящее учебно-методическое пособие ориентировано на обучающихся по направлению подготовки 01.03.02 «Прикладная математика и информатика», изучающих курс «Операционные системы». Пособие включает в себя описание четырех лабораторных работ, а также необходимые теоретические сведения, список литературы и приложение с формой отчета о выполнении лабораторной работы.

Цель пособия — привить студентам навыки решения типовых задач, которые возникают в практике программиста, разрабатывающего компоненты операционных систем.

Исчерпывающая информация по теории построения операционных систем, а также все необходимые для выполнения лабораторных работ теоретические сведения изложены в книге Э. Таненбаума и Х. Боса «Современные операционные системы» [1]. Особенности реализации сетевых операционных систем подробно рассмотрены в учебнике [2]. Многие аспекты основ программирования в операционных системах семейства GNU/Linux, изложенные в учебном пособии [3], окажутся полезными и при низкоуровневой разработке в операционных системах семейства BSD. Информацию об архитектуре компьютера, знание которой является ключевым для понимания основ функционирования операционных систем, можно получить из книги [4].

В приложении приведена форма отчета о выполнении лабораторной работы по курсу «Операционные системы». Такая структура отчета позволяет сделать акцент на наиболее важных моментах лабораторной работы и формирует привычку обращения к современной научно-технической документации.

Структура изложения материала, постановка задач и требования к написанию отчетов основаны на методических указаниях [5], много лет используемых в учебной работе на кафедре ИУ9.

Каждая лабораторная работа разбита на две части, задания в них практически одинаковы, но в практической реализации

опираются на разные операционные системы (ОС). Это сделано для того, чтобы студенты имели представление о разработке как Windows-подобных операционных систем (например, Microsoft Windows 10), так и UNIX-подобных ОС (например, Apple Mac OS X). Используемые для выполнения лабораторных работ операционные системы (аналог Windows — ReactOS и UNIX-подобная ОС NetBSD, поддерживаемая сообществом) являются бесплатными, и их исходный код открыт под *свободной* лицензией. Таким образом, студенты получают навыки разработки ОС двух основных и наиболее распространенных в настоящее время типов.

Во всем многообразии существующего программного обеспечения (ПО) качеству кода ОС уделяется особое внимание. К нему предъявляются наиболее высокие требования, поскольку от производительности, стабильности и безопасности (в самом широком смысле) работы операционной системы зависят характеристики и потребительские свойства всей вычислительной платформы и прикладных программ, которые выполняются на ее основе. Операционные системы со свободно распространяемым исходным кодом тестируются и совершенствуются широким кругом лиц и заинтересованных организаций, благодаря чему их код может послужить для студентов примером лучших практик разработки ПО.

Разработка ОС широким кругом авторов накладывает определенные требования на организацию исходных текстов (структуру каталогов, именование файлов и сущностей, способы передачи данных между функциональными частями ОС и др.) таким образом, чтобы они оставались понятными большинству разработчиков в течение многих лет. В силу этого исходный код свободно распространяемых операционных систем являет собой пример подхода к организации и структурированию больших проектов.

Лабораторные работы следуют друг за другом в порядке возрастания сложности. Поскольку в типичном случае для реализации компонентов ОС требуются написание и отладка больших объемов кода, лабораторные работы имеют преемственность: для выполнения очередного задания необходимо использование уже полученных практических результатов. Такой подход, с одной стороны, позволяет сократить затраты времени на выполнение лабораторных работ. С другой стороны, необходимость повторного использования кода собственной разработки формирует у студентов ответственное отношение к проектированию, программированию и отладке приложений.

КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Здесь приведены краткие сведения об ОС, их функциях, структуре и составных частях. Представленный материал не является исчерпывающим кратким курсом дисциплины. Его основное назначение — определение терминов, обязательных к использованию при выполнении лабораторных работ, составлении отчетов и в дальнейшей практической работе. Вместе с тем приводится также ключевая техническая информация о компонентах ОС и их взаимодействии с устройствами вычислительной машины. Таким образом, применительно к данному пособию этот материал выполняет функцию справочного.

Определение операционной системы

Современные компьютеры всегда оснащаются специальным программным обеспечением, называемым *операционной системой*. В ее задачи входят управление другими программами, ресурсами и выполнение дополнительных операций. ОС во многом отличается от пользовательских программ, например, ОС напрямую работает с аппаратным обеспечением, своим местоположением и др. В научной литературе операционные системы определяют по-разному, но можно выделить два основных определения:

- 1) операционная система как расширенная машина;
- 2) операционная система как менеджер ресурсов.

Первое определение ОС в качестве *расширенной машины* основывается на подходе «сверху вниз», при этом подразумевается следующее. Архитектура большинства компьютеров довольно разнообразна и недостаточно удобна для непосредственного применения в пользовательских программах. Так, простейшая задача чтения/записи данных будет необычайно сложной, если реализовывать

прямой доступ к жесткому диску с использованием спецификации стандарта SATA для взаимодействия с контроллером. Вместо этого операционная система предоставляет абстрактный интерфейс для чтения и записи блоков на диске, именуемый *драйвером диска поблочного доступа*. Далее перед пользовательским приложением возникает вопрос, какой блок прочитать и какой считать. В этом случае ОС предоставляет еще один интерфейс — файловую систему, с которой работает большинство пользовательских программ и которая, в свою очередь, работает с драйверами дисковых устройств. Такого рода абстракции являются важнейшим механизмом операционных систем. Фактически одной из главных задач ОС является создание удобных, неизменных абстракций вместо прямого доступа к аппаратному обеспечению.

Второе определение ОС в качестве *менеджера ресурсов* основывается на подходе «снизу вверх». Современный компьютер представляет собой сложную систему, состоящую из процессора, памяти, дисков, периферийных и прочих устройств. Задача операционной системы заключается в управлении всеми этими устройствами для обеспечения упорядоченного распределения процессорного времени, памяти, устройств ввода-вывода между различными программами, их использующими. Это особенно важно в современных ОС, в которых допускается (и даже необходима) одновременная работа нескольких программ. То же касается работы не только с аппаратной частью (сеть, управление и защита памяти), но и с информацией (файлы, базы данных и т. п.).

Управление ресурсами включает в себя распределение во времени и в пространстве. В случае разделения *во времени* ресурс используется различными программами по очереди. Например, программы исполняются на центральном процессоре в некоей последовательности, которую определяет операционная система. В случае *пространственного* разделения ресурсов вместо поочередной работы каждому потребителю предоставляется некоторая часть ресурса. Так, оперативная память делится несколькими программами, поскольку эффективнее разместить в памяти сразу все программы, чем загружать каждую, выполнять ее и затем выгружать (конечно, если памяти достаточно для размещения всех программ). Другим подобным ресурсом является жесткий диск. Распределение дискового пространства и отслеживание его использования — типичная задача ОС по управлению ресурсами.

Аппаратное обеспечение компьютера

Любой компьютер состоит как минимум из одного или из нескольких центральных процессоров, нескольких видов памяти и устройств ввода-вывода. Эти устройства объединены различными шинами.

Центральный процессор выбирает команды из памяти и выполняет их. Внутри процессора на каждом такте происходят выборка команды из памяти, ее декодирование для определения типа и операндов и непосредственно выполнение. Для каждого типа процессора существует определенный набор команд, которые не совместимы с другими процессорами. Например, процессор типа ARM не может исполнять команды процессора x86, и наоборот. Во всех современных процессорах есть регистры общего назначения, предназначенные для хранения очень небольшого количества данных. Часто это промежуточные данные или результат выполнения команды. В дополнение к регистрам общего назначения у процессоров есть несколько специальных регистров:

- *счетчик команд* (program counter, PC), который содержит адрес ячейки памяти со следующей командой для выполнения. После исполнения команды счетчик команд увеличивается;
- *указатель стека* (stack pointer, SP), который ссылается на вершину стека в памяти;
- *слово состояния программы* (program status word, PSW), который содержит различные управляющие биты, такие как биты управления приоритетом центрального процессора, режимом (пользователя или ядра) и др.

Большинство центральных процессоров (за исключением самых простых для встраиваемых систем) имеют как минимум два режима работы: режим ядра и режим пользователя. При работе в режиме *ядра* процессор может выполнять любые команды и использовать любые возможности аппаратуры без ограничений. В *пользовательском* режиме обычно исполняются пользовательские программы, а в некоторых видах операционных систем (таких как микроядерные ОС) в нем выполняются и части ядра операционной системы. В этом режиме на уровне центрального процессора есть ограничения на выполнение определенных операций, включена защита памяти, а также ограничена запись в некоторые регистры, такие как, например, PSW.

Пользовательская программа может обратиться к операционной системе только одним способом, который называется *системным вызовом*. В момент осуществления системного вызова происходит смена режима процессора с режима пользователя на режим ядра, а после завершения выполнения системного вызова, наоборот, — с режима ядра на режим пользователя.

Память является второй основной составляющей современного компьютера. Существует несколько разных типов памяти, которая используется в соответствии с ее характеристиками. Самая быстрая память — регистры центрального процессора, затем идет кэш-память процессора, затем основная память (также именуемая оперативным запоминающим устройством, ОЗУ); самой медленной, но самой емкой является долговременная память на магнитных и твердотельных накопителях. С точки зрения операционной системы важными типами памяти являются основная и долговременная, которыми ОС активно управляет и которые задействуют все пользовательские программы. В дополнение к ОЗУ многие компьютеры оснащены так называемой неизменяемой памятью — постоянным запоминающим устройством (ПЗУ); она предназначена только для чтения и является энергонезависимой. Обычно в такой памяти размещаются загрузчик, программы управления контроллерами ввода-вывода.

Большинство компьютеров и операционных систем поддерживают механизм работы с памятью, который называется *виртуальной памятью*. Она дает возможность запускать программы, по объему превышающие физическую память, за счет использования дисковой памяти для хранения редко привлекаемых данных. Для реализации виртуальной памяти требуется прозрачное для программы преобразование адресов памяти из физического в виртуальный и наоборот. Такое отображение осуществляется специальным модулем центрального процессора, называемым *блоком управления памятью* (memory management unit, MMU) и *диспетчером памяти* операционной системы.

Другой тип ресурсов компьютера, которым должна управлять операционная система, — устройства ввода-вывода информации. Обычно такие устройства состоят из двух компонентов: собственно устройства и его контроллера. Поскольку все типы контроллеров отличаются друг от друга, для управления ими нужно различное программное обеспечение. Такое программное обеспечение

называется *драйвером устройства*. Каждый производитель устройств обычно вместе с устройством поставляет драйверы под ту или иную ОС. Большинство драйверов работает в режиме ядра, однако существуют драйверы, работающие в режиме пользователя. Основной проблемой является то, что драйверам, работающим в режиме пользователя, нужно некоторым способом получать доступ к устройству, которым они управляют.

В результате увеличения скорости работы процессоров и объемов памяти возможностей одной шины стало недостаточно. Появились дополнительные шины для более быстрых устройств ввода-вывода, для обмена данными между центральным процессором и памятью. ОС должна знать обо всех шинах в компьютере и осуществлять их управление и настройку. Основной шиной на сегодняшний день является интерфейс периферийных устройств (Peripheral Component Interconnect, PCI), существует также множество дополнительных шин: USB, SATA, DMI, PCIe, SCSI.

Основные понятия операционной системы

Для многих ОС вне зависимости от особенностей их реализации используются определенные понятия и абстракции. Рассмотрим кратко основные из них.

Процесс — ключевое понятие во всех операционных системах. Процессом является программа во время ее выполнения. У каждого процесса есть свое *адресное пространство*. Это область памяти от нуля до некоторого максимума, с которой может взаимодействовать (читать, писать) процесс, связанный с этим адресным пространством. Также с каждым процессом связан набор ресурсов, например, список открытых файлов, регистры процессора, список связанных процессов и пр. Отсюда вытекает еще одно определение процесса: это контейнер, содержащий всю информацию, необходимую для работы программы. Во многих ОС информация о каждом процессе, кроме содержимого его адресного пространства, хранится в специальной таблице операционной системы, называемой *таблицей процессов*, в виде массива или связанного списка структур. *Образом памяти* называют содержимое адресного пространства процесса. В том случае, если в операционной системе предусмотрен механизм создания процессом новых процессов,

такие процессы называются *дочерними* по отношению к создавшему их *родительскому* процессу. Обмен данными между процессами и взаимная синхронизация их действий называются *межпроцессным взаимодействием*.

Файлы — еще одно ключевое понятие ОС. Как указано выше, основной функцией ОС является сокрытие специфики дисков и других устройств ввода-вывода и предоставление разработчику более удобной абстрактной модели, состоящей из независимых от устройств файлов. Для работы с файлами (создание, удаление, чтение, запись) операционная система должна предоставлять соответствующие системные вызовы.

Файлы объединяются в группы посредством *каталога* (или *директории*). Элементами каталога могут быть файлы или другие каталоги. Каталоги и файлы образуют иерархическую структуру файловой системы. Каждый файл может быть обозначен своим *полным именем* с указанием пути к файлу. *Абсолютный путь* состоит из списка каталогов, которые нужно пройти от *корневого каталога*, чтобы добраться до файла. В качестве разделителя имен каталогов используется прямая или обратная косая черта (слэш). У каждого процесса в любой момент времени есть текущий *рабочий каталог*, относительно которого рассматриваются пути файлов, не начинающиеся со слэша. Работа с файлами организована так, что сначала необходимо открыть файл, а после окончания всех действий его закрыть. В результате открытия файла по его полному (или относительному) пути и имени операционная система возвращает пользователю *дескриптор* файла. Все последующие операции с этим файлом, включая закрытие, осуществляются с использованием этого дескриптора.

Системные вызовы

Выше было указано, что операционные системы предоставляют абстракции пользовательским программам и управляют ресурсами компьютера. Интерфейс между пользовательскими программами и операционной системой строится на этих абстракциях.

Системный вызов — это механизм, предоставляемый ОС, который позволяет программе запрашивать услугу или ресурс у ОС. Когда программе необходимо получить доступ к ресурсу или выполнить

задачу, требующую привилегированного доступа или выходящую за рамки ее области, она выполняет системный вызов ОС.

Системные вызовы обеспечивают интерфейс между программами пользовательского пространства и ядром ОС. *Ядро* — это центральный компонент ОС, который управляет системными ресурсами и предоставляет услуги программам пользовательского пространства.

Примерами системных вызовов служат открытие и закрытие файлов, чтение файлов и запись данных в них и из них, создание процессов и управление ими, выделение и освобождение памяти, а также управление устройствами ввода-вывода.

Когда программа выполняет системный вызов, она передает управление ядру, которое исполняет запрошенную операцию от имени программы. Как только ядро завершило операцию, оно возвращает управление программе, которая может продолжить работу.

Архитектура операционной системы

Архитектура ОС описывает как организованы компоненты ОС и как они взаимодействуют друг с другом. На самом высоком уровне ОС можно разделить на два основных компонента: ядро и пользовательское пространство. Как было указано, *ядро* является основной частью ОС и отвечает за управление системными ресурсами, такими как центральный процессор, память и устройства ввода-вывода. *Пользовательское пространство* — это часть ОС, в которой запускаются пользовательские приложения. Пользовательское пространство взаимодействует с ядром через системные вызовы.

Ядро можно разделить на несколько компонентов, включая управление процессами, управление памятью, файловую систему и драйверы устройств. Управление процессами отвечает за выполнение пользовательских приложений, включая планирование и синхронизацию. Управление памятью отвечает за выделение и освобождение ресурсов памяти. Файловая система предоставляет пользователям интерфейс для доступа к файлам и каталогам и управления ими. Драйверы устройств отвечают за управление связью с аппаратными устройствами, включая драйверы сетевых карт, звуковых карт и принтеров.

Существуют две основные архитектуры ОС — *монолитная* и *микроядерная*.

В монолитном ядре все компоненты ОС работают в режиме ядра, а ядро обеспечивает прямой доступ к аппаратным ресурсам. Эта архитектура проста и эффективна, но ее сложно поддерживать и расширять, так как любое изменение ядра требует полной перекомпиляции ОС. Примеры ОС, использующих монолитное ядро, — Linux и Windows.

Архитектура микроядра, напротив, разделяет компоненты ОС на разные процессы, и только небольшая часть ядра обеспечивает доступ к аппаратным ресурсам. Эта архитектура более модульная и гибкая вследствие большей простоты процесса добавления или удаления компонентов, который не затрагивает всю ОС. Однако производительность ОС на основе микроядра может быть ниже, чем у ОС на основе монолитного ядра, поскольку между ядром и пользовательским пространством возникает больше «накладных расходов». В качестве примера ОС, использующих микроядерную архитектуру, можно привести QNX и Minix.

Другой архитектурой, получившей популярность в последние годы, является архитектура *гибридного ядра*. В гибридном ядре одни компоненты ОС работают в режиме ядра, другие — в режиме пользователя. Эта архитектура сочетает в себе преимущества монолитной и микроядерной архитектур: она более гибкая и модульная, чем монолитное ядро, но с более высокой производительностью, чем микроядро. В качестве примера операционных систем, использующих архитектуру гибридного ядра, можно привести Mac OS и семейство операционных систем Windows NT.

В дополнение к традиционным архитектурам ОС существует несколько новых архитектур, которые набирают популярность, например, архитектуры на основе *виртуальных машин* и на основе *контейнеров*.

В архитектуре на основе виртуальных машин гипервизор работает в основной ОС, и поверх гипервизора можно создать несколько виртуальных машин. Каждая виртуальная машина имеет свою собственную ОС, которая может отличаться от ОС хоста. Эта архитектура обеспечивает лучшую изоляцию между приложениями, поскольку каждая виртуальная машина независима от других. Виртуальные машины можно перемещать между разными физическими машинами, что обеспечивает большую гибкость и мас-

штабируемость. Примеры архитектур на основе виртуальных машин — VMware и VirtualBox.

В архитектуре на основе контейнеров несколько контейнеров работают поверх одной ОС хоста, каждый контейнер содержит пользовательское приложение и его зависимости. Контейнеры обеспечивают лучшую изоляцию, чем традиционные ОС, но с меньшими «накладными расходами», чем виртуальные машины, поскольку они используют одно и то же ядро ОС. Контейнерные архитектуры становятся все более популярными для запуска веб-приложений и микросервисов. Примеры архитектур на основе контейнеров — Docker и Kubernetes.

Наконец, важно отметить, что архитектура ОС может оказывать существенное влияние на ее безопасность. Монолитные ядра могут быть более уязвимы для атак, так как любая уязвимость в ядре влияет на всю систему. Архитектуры на основе микроядра могут обеспечить бо́льшую безопасность, поскольку только малая часть ядра имеет прямой доступ к аппаратным ресурсам. Контейнерные архитектуры также могут обеспечить бо́льшую безопасность, поскольку каждый контейнер изолирован от других и может обращаться только к своим собственным ресурсам.

Архитектура ОС играет большую роль в ее производительности, надежности и безопасности. Традиционные архитектуры ОС, такие как монолитная и микроядерная, по-прежнему широко используются, но набирают популярность более новые подходы, такие как виртуализация и контейнеризация. Каждая архитектура имеет свои сильные и слабые стороны, и выбор архитектуры зависит от конкретных требований к разрабатываемой системе.

ТРЕБОВАНИЯ К ВЫПОЛНЕНИЮ ЛАБОРАТОРНЫХ РАБОТ

Для выполнения лабораторных работ понадобится персональный компьютер распространенной архитектуры с не менее чем четырехядерным процессором и тактовой частотой не ниже 1 ГГц, имеющий ОЗУ не менее 4 Гб и свободное место на жестком диске не менее 200 Мб, работающий под управлением ОС Windows, Linux или Mac OS. Выполнение лабораторных работ на ПК с более низкими характеристиками возможно, но будет затруднительным и некомфортным, так как предполагаются компиляция больших объемов кода и запуск изучаемой ОС в среде гипервизора, т. е. выполнение двух ОС с сопутствующим ПО на одном ПК одновременно. Наличие средств аппаратной поддержки виртуализации в архитектуре процессора ПК дает значительное увеличение скорости выполнения гостевых ОС.

Рекомендуемый программный продукт для виртуализации (гипервизор) — VirtualBox компании Oracle. Этот гипервизор является бесплатным и доступен для скачивания по ссылке <https://www.virtualbox.org/wiki/Downloads>, что является несомненным плюсом в процессе обучения. При желании возможно использование других программных продуктов, например VMWare. Программные средства, необходимые для выполнения конкретных лабораторных работ, приведены в их описаниях.

Любая операционная система включает некоторый набор средств отладки. Одним из основных таких инструментов являются журналы сообщений и событий. В рамках выполнения лабораторных работ по курсу «Операционные системы» основным средством отладки будет журнал системных сообщений. Далее в тексте для обозначения журнала системных сообщений дополнительно используется словосочетание `debug log` (отладочный лог, отладочная информация).

Все вопросы, связанные с отладкой ОС ReactOS, подробно рассмотрены на официальном сайте проекта в разделе, посвященном

отладке и доступном по ссылке <https://reactos.org/wiki/Debugging>. Например, отладочный лог в режиме реального времени можно получать, направив его вывод в последовательный порт и, в свою очередь, перенаправив вывод из последовательного порта в гипервизоре в именованный канал (named pipe).

Рекомендуемым способом отладки ПО, разработанного в ходе выполнения лабораторной работы, является перенаправление вывода последовательного порта в файл в ОС хоста (но не в гостевую ОС, которая и отлаживается!) с его последующим просмотром. Некоторые программы обновляют содержимое файла в реальном времени, что позволяет наблюдать за отладочными сообщениями без необходимости в ручном обновлении содержимого файла. В такой режим можно настроить, например, программу Notepad++, включив опции Update silently и Scroll to the last line after update в меню Preferences. Более подробно ознакомиться с различными способами автоматического обновления файла можно в популярном среди системных разработчиков блоге Raymond, перейдя по ссылке <https://www.raymond.cc/blog/monitor-log-or-text-file-changes-in-real-time-with-notepad/>.

Всего в курсе предусмотрено выполнение четырех лабораторных работ. Каждая работа состоит из двух частей, задания в них одинаковы, но используются разные операционные системы — бесплатные, свободно распространяемые ОС с открытым исходным кодом: ReactOS и NetBSD. Первая ОС позволяет отточить навыки разработки под ОС семейства Microsoft Windows, используя ряд преимуществ, а именно гораздо более быструю установку системы, меньшие системные требования и наличие исходных кодов. ОС NetBSD дает возможность ознакомиться с навыками разработки под управлением Unix-подобных ОС без излишне больших системных требований. Результаты выполнения каждой части лабораторной работы можно демонстрировать преподавателю отдельно.

ПРАКТИЧЕСКАЯ ЧАСТЬ

Работа № 1. ReactOS и NetBSD. Среда сборки, установка и тестирование в виртуальной машине

Цель работы — приобретение навыков самостоятельной сборки «с нуля» дистрибутивов ОС с последующей их установкой в виртуальную машину. В первой части работы используется ОС ReactOS, во второй части — NetBSD. В последующих лабораторных работах порядок тот же.

Часть 1. Сборка дистрибутива ReactOS

Постановка задачи

Настроить среду сборки и тестирования ReactOS на своем компьютере (или на компьютере в учебной аудитории Университета), собрать установочный образ и произвести его установку в виртуальную машину с помощью программных средств виртуализации.

Порядок выполнения работы

Для выполнения лабораторной работы необходимо реализовать следующие этапы.

1. Получить среду сборки ReactOS Build Environment (RosBE) по ссылке со страницы https://reactos.org/wiki/Build_Environment и установить ее согласно прилагаемому к ней руководству.
2. Установить систему управления версиями Git (например, TortoiseGit + MSysGit для Windows с сайта <https://tortoisegit.org> или с помощью менеджера пакетов в случае использования Unix-подобных ОС).
3. Создать у себя рабочую копию (working copy) исходных кодов основной ветки разработки master из репозитория <https://github.com/reactos/reactos.git>.
4. Произвести сборку установочного ISO-образа операционной системы ReactOS, используя подробное руководство со страницы https://reactos.org/wiki/Building_ReactOS.

5. Установить полученный образ в виртуальной машине (VirtualBox), настроив вывод отладочной информации (debug log) в файл, используя инструкции с сайта <https://reactos.org/wiki/Debugging>, а также приведенные выше рекомендации.

Модифицировать ядро операционной системы (ntoskrnl) так, чтобы в отладочный лог в момент инициализации ядра выводилась фамилия студента, выполнившего лабораторную работу. Место для вставки кода следует выбирать произвольно путем изучения дерева исходного кода ядра, отладочного лога, что является показателем самостоятельного выполнения работы студентом.

6. Заменить ядро в установленной ОС в виртуальной машине на модифицированное и загрузить снова, убедиться в наличии нужной информации в debug log.

Результаты

Преподавателю демонстрируется следующее.

1. Компьютер с установленными средствами сборки RosBE, системой контроля версий Git, системой виртуализации VirtualBox в основной ОС (host OS).

2. Виртуальная машина с установленной гостевой ОС ReactOS из дистрибутива, собранного самостоятельно из исходного кода системы.

3. Отладочный лог, в котором на каком-либо этапе инициализации ядра выводится фамилия студента, выполнившего работу.

Часть 2. Операционная система NetBSD: среда сборки, установка и тестирование

Постановка задачи

Установить новый выпуск NetBSD из дистрибутива с официального сайта проекта NetBSD <https://www.netbsd.org/> в виртуальную машину. Настроить среду сборки и тестирования NetBSD в виртуальной машине на своем компьютере (или на компьютере в учебной аудитории Университета). Пересобрать ядро в этой виртуальной машине, добавив вывод фамилии обучающегося в debug log.

Важно! Не следует предпринимать попытки кросс-компилировать ядро NetBSD в Windows, Linux или OS X. Собирать его нужно внутри самой NetBSD, которая установлена в виртуальной машине.

Порядок выполнения работы

Для выполнения лабораторной работы необходимо реализовать следующие этапы.

1. Скачать с сайта <https://netbsd.org/> образ дистрибутива ОС NetBSD и установить его в виртуальную машину (VirtualBox), настроив вывод отладочной информации (debug log) в файл.
2. Настроить среду сборки и скачать исходный код системы.
3. Модифицировать ядро операционной системы так, чтобы в отладочный лог в момент инициализации ядра выводилась фамилия студента, выполнившего лабораторную работу.
4. Заменить ядро в установленной ОС в виртуальной машине на модифицированное и загрузить снова, убедиться в наличии нужной информации в debug log.

Результаты

Преподавателю демонстрируется следующее.

1. Виртуальная машина с установленной гостевой ОС NetBSD из дистрибутива, скачанного с официального сайта <https://netbsd.org/>.
2. Установленные пакеты и исходный код, необходимый для перекомпиляции ядра NetBSD.
3. Отладочный лог загрузки ОС, в который на каком-либо этапе инициализации ядра происходит вывод фамилии студента, выполнившего лабораторную работу.

Вопросы для защиты

1. Какая команда используется для сборки образа установочного компакт-диска?
2. Каким образом можно собрать один или несколько модулей, например только ядро?
3. Какими способами можно заменить файлы в установленной в виртуальной машине ОС?

Работа № 2. Загружаемый модуль ядра (драйвер)

Цель работы — разработать простой загружаемый модуль ядра (драйвер), который выводит в отладочный журнал фамилию студента, выполнившего лабораторную работу. Драйвер должен компилироваться в отдельный файл, а не быть частью ядра.

Часть 1. Драйвер в ReactOS

Постановка задачи

В созданном в лабораторной работе № 1 рабочем дереве ОС ReactOS создать новый модуль, который представляет собой простейший драйвер, совместимый с ОС Windows NT и ReactOS. Драйвер должен реализовать минимальный набор функций, необходимый для загрузки и выгрузки этого драйвера. В функции инициализации этого драйвера DriverEntry осуществить вывод в отладочный лог, используя макрос DPRINT1, фамилии студента, выполнившего лабораторную работу.

Порядок выполнения работы

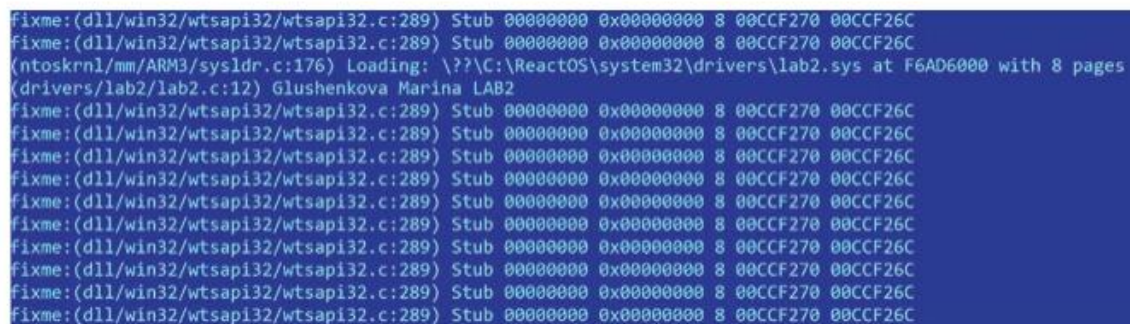
1. Перейти в дерево исходного кода ReactOS, полученного в результате выполнения лабораторной работы № 1.
2. Создать в папке drivers новую папку с названием, например, lab2.
3. Добавить в конец файла drivers\CMakeLists.txt запись `add_subdirectory(lab2)`, для того чтобы папка lab2 также участвовала в сборке системы.
4. В папку drivers\lab2 поместить три файла:
 - CMakeLists.txt, который будет информировать систему сборки о том, что именно и каким именно образом должно быть скомпилировано и слинковано (так называемые *правила сборки модуля*);
 - lab2.c — файл с исходным кодом драйвера;
 - lab2.rc — файл с информацией о драйвере, который используется ReactOS и Windows.
5. Собрать новый установочный образ операционной системы с помощью команды `ninja bootcd`.
6. Установить операционную систему из полученного образа в виртуальную машину.
7. В работающей системе в виртуальной машине выполнить загрузку драйвера по требованию с помощью следующих команд:

```
sc create lab2driver binPath= C:\ReactOS\system32\drivers\lab2.sys type= kernel
sc start lab2driver
```

Результаты

Преподавателю демонстрируется следующее.

1. Настроенная среда сборки ReactOS, включающая новый модуль — драйвер.
2. Виртуальная машина с установленной гостевой ОС ReactOS из самостоятельно созданного ISO-образа, включающего разработанный драйвер.
3. В загруженной ОС ReactOS в виртуальной машине при вводе команды `sc start lab2driver` разработанный драйвер выводит в отладочный лог фамилию студента, выполнившего лабораторную работу (рис. 1).



```
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
(ntoskrnl/mm/ARM3/sysldr.c:176) Loading: \??\C:\ReactOS\system32\drivers\lab2.sys at F6AD6000 with 8 pages
(drivers/lab2/lab2.c:12) Glushenkova Marina LAB2
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
fixme:(dll/win32/wtsapi32/wtsapi32.c:289) Stub 00000000 0x00000000 8 00CCF270 00CCF26C
```

Рис. 1. Пример вывода в виде снимка экрана

Часть 2. Драйвер в NetBSD

Постановка задачи

В виртуальной машине с NetBSD, созданной в лабораторной работе № 1, создать новый загружаемый модуль ядра (loadable kernel module), реализующий простейший драйвер. Драйвер должен содержать минимальный набор функций, необходимый для загрузки и выгрузки этого драйвера. В функции инициализации драйвера осуществить вывод в отладочный лог фамилии студента, выполнившего лабораторную работу.

Список загружаемых драйверов можно получить командой `find /stand -type f`, а затем найти код искомого модуля.

Порядок выполнения работы

1. Создать файл `/usr/src/sys/dev/lab2.c`.

2. Добавить минималистичную реализацию драйвера:

```
#include <sys/module.h>
MODULE(MODULE_CLASS_MISC, lab2, NULL);
static int lab2_modcmd(modcmd_t cmd, void* arg) {
    printf("driver LAB2");
    return 0;
}
```

3. Создать файл /usr/src/sys/modules/lab2/Makefile:

```
.include "../Makefile.inc"
KMOD=lab2
.PATH: ${S}/dev
SRCS=lab2.c
.include <bsd.kmodule.mk>
```

4. Выполнить команду make.

5. Выполнить команду modload ./lab2.kmod.

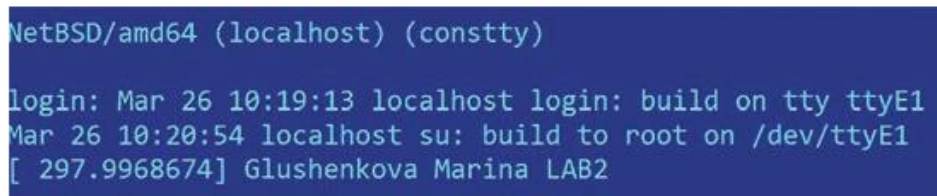
Более подробное руководство по разработке драйверов для операционной системы NetBSD можно открыть, перейдя по ссылке <http://www.home.unix-ag.org/bmeurer/NetBSD/howto-lkm.html>.

Результаты

Преподавателю демонстрируется следующее.

1. Виртуальная машина с установленной гостевой операционной системой NetBSD и настроенной системой сборки.

2. В загруженной ОС NetBSD в виртуальной машине при вводе команды modload ./lab2.kmod в отладочный лог выводится фамилия студента, выполнившего лабораторную работу (рис. 2).



```
NetBSD/amd64 (localhost) (constty)
login: Mar 26 10:19:13 localhost login: build on tty ttyE1
Mar 26 10:20:54 localhost su: build to root on /dev/ttyE1
[ 297.9968674] Glushenkova Marina LAB2
```

Рис. 2. Пример вывода в виде снимка экрана

Вопросы для защиты

1. Какая команда используется для сборки модуля ядра (драйвера)?
2. Как можно вывести список всех загруженных драйверов в системе?
3. Есть ли возможность выгрузить ранее загруженный драйвер?
4. Какие различия есть в загружаемых модулях ядра ReactOS и NetBSD?
5. Каковы основные типы загрузки драйверов?

Работа № 3. Список процессов в системе

Цель работы — разработать загружаемые модули ядра (драйвера) для ОС Windows или ReactOS и NetBSD, которые выводят в отладочный журнал список всех процессов в системе.

Часть 1. Вывод списка процессов из драйвера в ReactOS

Постановка задачи

На основе драйвера из лабораторной работы № 2 создать новый драйвер, который выводит в отладочный лог список процессов в системе, используя API ядра. Список должен состоять из имени процесса (имя образа — Image Name), идентификатора процесса (PID) и идентификатора родительского процесса.

Драйвер должен загружаться и выгружаться по требованию так, чтобы можно было проверить соответствие списков процессов, выведенных драйвером, и с помощью утилиты Task Manager (taskmgr.exe).

Информацию о процессах получить из ядра с использованием функции ZwQuerySystemInformation(SystemProcessInformation, ...). Выделение и освобождение памяти для информации о процессах реализовать, используя функции ExAllocatePool(PagedPool, ...) и ExFreePool(). Предусмотреть ситуацию, в которой запрошенный размер буфера, необходимый для размещения информации о процессах, может измениться в большую сторону между вызовами функции ZwQuerySystemInformation, и в этом случае произвести новое выделение памяти с освобождением старого буфера.

Порядок выполнения работы

1. Перейти в дерево исходного кода ReactOS, полученного в результате выполнения лабораторной работы № 1.

2. Создать в папке drivers папку с названием, например, lab3, содержимое которой является копией папки с драйвером из лабораторной работы № 2 (lab2) с переименованными файлами lab2.c и lab2.rc в lab3.c и lab3.rc соответственно

3. Добавить в конец файла drivers\CMakeLists.txt запись add_subdirectory(lab3), для того чтобы папка lab3 также участвовала в сборке системы.

4. В папке drivers\lab3 отредактировать файл со сборочной информацией CMakeLists.txt, изменив там названия исходных файлов.

5. Реализовать в файле lab3.c вывод списка процессов. Для корректного определения структуры SYSTEM_PROCESS_INFORMATION включить следующие заголовочные файлы:

```
#include <ntddk.h>
#include <ntifs.h>
#include <ndk/exfuncs.h>
#include <ndk/ketypes.h>
#include <pseh/pseh2.h>
#include <ntstrsafe.h>
```

6. Собрать новый установочный образ ОС с помощью команды ninja bootcd.

7. Установить ОС из полученного образа в виртуальную машину.

8. В работающей системе в виртуальной машине выполнить загрузку драйвера по требованию с помощью следующих команд:

```
sc create lab3driver binPath=C:\ReactOS\system32\drivers\
lab3.sys type= kernel
sc start lab3driver
```

Результаты

Преподавателю демонстрируется следующее.

1. Компьютер с установленными средствами сборки RosBE, системой контроля версий Git, системой виртуализации VirtualBox в основной ОС (host OS).

2. Виртуальная машина с установленной гостевой ОС ReactOS из дистрибутива, собранного самостоятельно из исходного кода системы.

3. В загруженной до рабочего стола ОС ReactOS в виртуальной машине при вводе команды sc start lab3driver в отладочный лог выводится список всех процессов в системе примерно следующего вида (точно соответствовать приведенным ниже строкам списка процессов могут только первые две строки, остальные идентификаторы, вероятно, будут отличаться):

```
(drivers/lab3/lab3.c:43) Process list:
0: (null)
4: System
108: smss.exe
```

156: csrss.exe
180: winlogon.exe
204: services.exe
212: lsass.exe
260: VBoxService.exe
284: svchost.exe
296: eventlog.exe
424: rpcss.exe
480: svchost.exe
528: explorer.exe
548: svchost.exe
600: wlansvc.exe
648: spoolsv.exe
760: VBoxTray.exe
768: kbswitch.exe
820: cmd.exe

Часть 2. Вывод списка процессов из драйвера в NetBSD

Постановка задачи

На основе драйвера из лабораторной работы № 2 создать новый драйвер, который выводит в отладочный лог список процессов в системе, используя API ядра. Список должен состоять из имени процесса (имя образа — Image Name), идентификатора процесса (PID) и идентификатора родительского процесса.

Драйвер должен загружаться и выгружаться по требованию так, чтобы можно было проверить соответствие списков процессов, выведенных драйвером, и с помощью утилиты ps.

Информацию о процессах получить из ядра, используя глобальную переменную allproc. Для этого необходимо включить заголовочные файлы:

```
#include <sys/module.h>  
#include <sys/kernel.h>  
#include <sys/proc.h>
```

Самостоятельно следует изучить проблему защиты от «грязного чтения» списка процессов в момент его изменения операционной системой (например, при создании нового процесса или удалении завершенного).

Порядок выполнения работы

1. Перейти в дерево исходного кода NetBSD, полученного в результате выполнения лабораторной работы № 1, и создать копию драйвера lab2 из лабораторной работы № 2 под названием lab3, внося соответствующие изменения.

2. Реализовать код для получения списка процессов в файле lab3.c. Для итерации по списку процессов использовать макрос PROCLIST_FOREACH.

3. Выполнить сборку загружаемого модуля ядра, вызвав команду make в директории /usr/src/sys/modules/lab3.

4. Выполнить загрузку драйвера по требованию с помощью следующей команды:

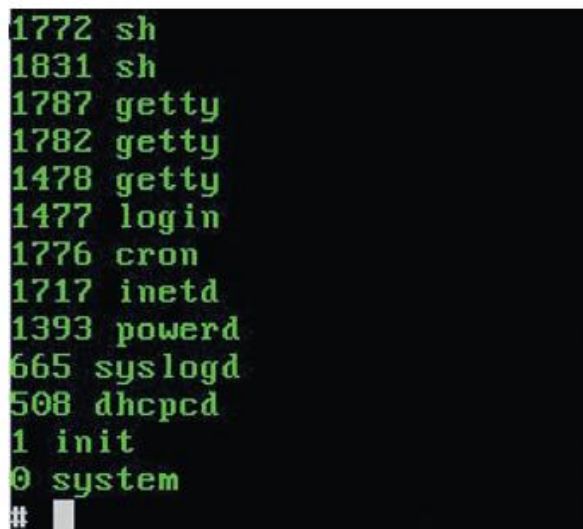
```
modload ./lab3.kmod
```

Результаты

Преподавателю демонстрируется следующее.

1. Виртуальная машина с установленной гостевой ОС NetBSD и настроенной системой сборки.

2. В загруженной ОС NetBSD в виртуальной машине при вводе команды modload ./lab3.kmod в отладочный лог выводится список всех процессов в системе. Примерное содержание списка процессов показано на рис. 3, но точно соответствовать ему могут только последние две строки, остальные идентификаторы, вероятно, будут отличаться.



```
1772 sh
1831 sh
1787 getty
1782 getty
1478 getty
1477 login
1776 cron
1717 inetd
1393 powerd
665 syslogd
508 dhcpd
1 init
0 system
#
```

Рис. 3. Пример вывода списка процессов драйвером в NetBSD

Вопросы для защиты

1. Что такое процесс с идентификатором 0 в системе ReactOS? Почему у него нет имени образа?
2. Что такое процессы с идентификаторами 0 и 1 в системе NetBSD? Есть ли отличие от ReactOS?
3. В чем различие PagedPool и NonPagedPool в ReactOS?
4. Почему нельзя использовать функцию ZwAllocateMemory вместо выделения памяти из пула?

Работа № 4. Операции с виртуальной памятью

Цель работы — разработать загружаемый по требованию модуль ядра (драйвер) для операционных систем Windows или ReactOS и NetBSD, работающий с виртуальной памятью.

Часть 1. Работа с памятью в режиме ядра в операционной системе семейства Windows NT

Постановка задачи

Используя созданный в лабораторной работе № 2 минимальный драйвер, совместимый с ОС Windows NT/ReactOS и NetBSD, реализовать следующие действия с виртуальной памятью.

1. Зарезервировать 10 страниц виртуальной памяти, используя функцию `ZwAllocateVirtualMemory(NtCurrentProcess(), MEM_RESERVE, ...)`.
2. Обеспечить пять первых страниц из выделенных 10 физическими страницами памяти, используя `ZwAllocateVirtualMemory(NtCurrentProcess(), MEM_COMMIT, ...)`.
3. Вывести физические адреса и значения PTE для всех 10 страниц в шестнадцатеричном формате.
4. Освободить выделенную память.

Справочную информацию по реализации можно дополнительно посмотреть в бюллетене, доступном для скачивания по ссылке https://doc.lagout.org/operating%20system%20/Windows/Memory_Scanning_Under_Windows_Nt.pdf.

Порядок выполнения работы

1. Перейти в дерево исходного кода ReactOS, полученного в результате выполнения лабораторной работы № 1.
2. Создать в папке `drivers` папку с названием, например, `lab4`, содержимое которой является копией папки с драйвером из лабораторной работы № 2 (`lab2`) с переименованными файлами `lab2.c` и `lab2.rc` в `lab4.c` и `lab4.rc` соответственно.
3. Добавить в конец файла `drivers\CMakeLists.txt` запись `add_subdirectory(lab4)`, для того чтобы папка `lab4` также участвовала в сборке системы.
4. В папке `drivers\lab4` отредактировать файл со сборочной информацией `CMakeLists.txt`, изменив там названия исходных файлов.
5. Реализовать код для выделения виртуального адресного пространства в файле `lab4.c` согласно постановке задачи.

6. Собрать новый установочный образ ОС с помощью команды `ninja bootcd`.

7. Установить ОС из полученного образа в виртуальную машину.

8. В работающей системе в виртуальной машине выполнить загрузку драйвера по требованию с помощью следующих команд:

```
sc create lab4driver binPath=
```

```
C:\ReactOS\system32\drivers\lab4.sys type= kernel
```

```
sc start lab4driver
```

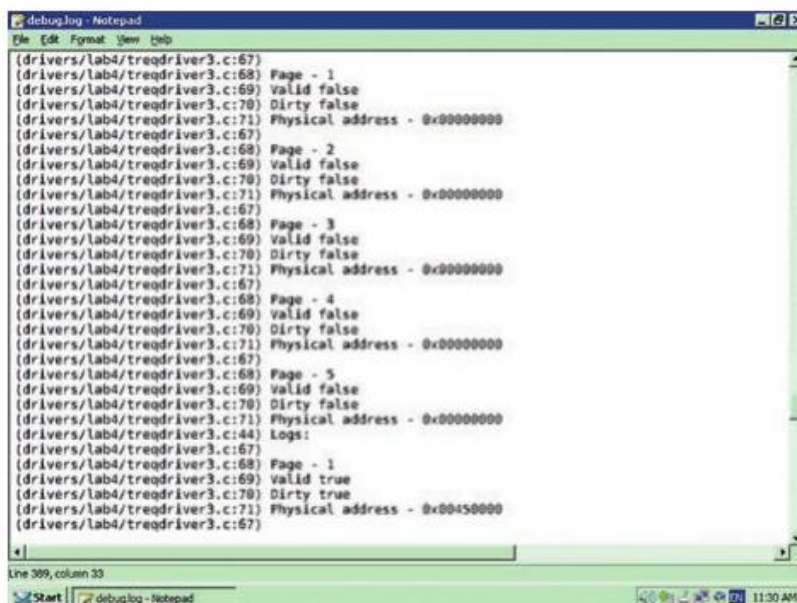
Результаты

Преподавателю демонстрируется следующее.

1. Компьютер с установленными средствами сборки RosBE, системой контроля версий Git, системой виртуализации VirtualBox в основной ОС (host OS).

2. Виртуальная машина с установленной гостевой ОС ReactOS из дистрибутива, собранного самостоятельно из исходного кода системы.

3. В загруженной до рабочего стола ОС ReactOS в виртуальной машине при вводе команды `sc start lab4driver` в отладочный лог выводится информация о выделенных виртуальных адресах и страницах физической памяти. Пример вывода из отладочного лога, открытого в редакторе Notepad, показан на рис. 4.



```
(drivers/lab4/treqdriver3.c:67)
(d/drivers/lab4/treqdriver3.c:68) Page - 1
(d/drivers/lab4/treqdriver3.c:69) Valid false
(d/drivers/lab4/treqdriver3.c:70) Dirty false
(d/drivers/lab4/treqdriver3.c:71) Physical address - 0x00000000
(d/drivers/lab4/treqdriver3.c:67)
(d/drivers/lab4/treqdriver3.c:68) Page - 2
(d/drivers/lab4/treqdriver3.c:69) Valid false
(d/drivers/lab4/treqdriver3.c:70) Dirty false
(d/drivers/lab4/treqdriver3.c:71) Physical address - 0x00000000
(d/drivers/lab4/treqdriver3.c:67)
(d/drivers/lab4/treqdriver3.c:68) Page - 3
(d/drivers/lab4/treqdriver3.c:69) Valid false
(d/drivers/lab4/treqdriver3.c:70) Dirty false
(d/drivers/lab4/treqdriver3.c:71) Physical address - 0x00000000
(d/drivers/lab4/treqdriver3.c:67)
(d/drivers/lab4/treqdriver3.c:68) Page - 4
(d/drivers/lab4/treqdriver3.c:69) Valid false
(d/drivers/lab4/treqdriver3.c:70) Dirty false
(d/drivers/lab4/treqdriver3.c:71) Physical address - 0x00000000
(d/drivers/lab4/treqdriver3.c:67)
(d/drivers/lab4/treqdriver3.c:68) Page - 5
(d/drivers/lab4/treqdriver3.c:69) Valid false
(d/drivers/lab4/treqdriver3.c:70) Dirty false
(d/drivers/lab4/treqdriver3.c:71) Physical address - 0x00000000
(d/drivers/lab4/treqdriver3.c:44) Logs:
(d/drivers/lab4/treqdriver3.c:67)
(d/drivers/lab4/treqdriver3.c:68) Page - 1
(d/drivers/lab4/treqdriver3.c:69) Valid true
(d/drivers/lab4/treqdriver3.c:70) Dirty true
(d/drivers/lab4/treqdriver3.c:71) Physical address - 0x00450000
(d/drivers/lab4/treqdriver3.c:67)
```

Рис. 4. Вывод отладочной информации

Следует обратить внимание на то, что для выделенных адресов виртуальной памяти, не обеспеченных физическими страницами, соответствующий адрес будет нулевым (первые пять страниц), а для следующих пяти страниц физический адрес будет соответствовать тем физическим страницам, на которые отображаются виртуальные адреса. Из флагов PTE особое внимание уделяется флагам Valid и Dirty. Как видно на рис. 4, для обеспеченных физическими страницами адресов эти флаги выставлены в True (потому что, во-первых, страница валидна, а во-вторых, туда произведена запись).

Часть 2. Работа с памятью в режиме ядра в NetBSD

Постановка задачи

Используя созданный в лабораторной работе № 2 драйвер, реализовать следующие действия с виртуальной памятью.

1. Зарезервировать 10 страниц виртуальной памяти, используя функцию ядра `uvm_km_alloc`.
2. Обеспечить пять первых страниц из выделенных 10 физическими страницами памяти, используя функции ядра `uvm_pglstalloc` и `rmap_kenter_pa`.
3. Вывести физические адреса и значения PTE для всех 10 страниц в шестнадцатеричном формате.
4. Освободить выделенную память, используя функции ядра `rmap_kremove`, `rmap_update`, `uvm_pglstfree`, `uvm_km_free`.

Драйвер должен выгружаться динамически с помощью команды `modunload`.

Порядок выполнения работы

1. Перейти в дерево исходного кода NetBSD, полученного в результате выполнения лабораторной работы № 1, и создать копию драйвера `lab2` из лабораторной работы № 2 под названием `lab4`, внося соответствующие изменения.
2. Реализовать код для выделения виртуального адресного пространства в файле `lab4.c`.
3. Выполнить сборку загружаемого модуля ядра, вызвав команду `make` в директории `/usr/src/sys/modules/lab4`.
4. Выполнить загрузку драйвера по требованию с помощью следующей команды:

```
modload ./lab4.kmod
```

Результаты

Преподавателю демонстрируется следующее.

1. Виртуальная машина с установленной гостевой ОС NetBSD и настроенной системой сборки.
2. В загруженной ОС NetBSD в виртуальной машине при вводе команды `modload ./lab4.kmod` в отладочный лог выводится информация о выделенной памяти (рис. 5).

```
[ 62.5460669] Page - 3
[ 62.5460669] Valid - false
[ 62.5460669] Used - false
[ 62.5460669] Modified - false
[ 62.5460669] Physical address - 0x00000000

[ 62.5460669] Page - 4
[ 62.5460669] Valid - false
[ 62.5460669] Used - false
[ 62.5460669] Modified - false
[ 62.5460669] Physical address - 0x00000000

[ 62.5460669] Page - 5
[ 62.5460669] Valid - false
[ 62.5460669] Used - false
[ 62.5460669] Modified - false
[ 62.5460669] Physical address - 0x00000000

[ 62.5460669] After commit:
[ 62.5460669] Page - 1
[ 62.5460669] Valid - true
[ 62.5460669] Used - false
[ 62.5460669] Modified - false
[ 62.5460669] Physical address - 0x02721000
```

Рис. 5. Пример вывода виртуальных адресов, обеспеченных и не обеспеченных физической памятью

Вопросы для защиты

1. Почему физические адреса страниц идут непоследовательно друг за другом?
2. Что такое буфер ассоциативной трансляции (Translation Lookaside Buffer, TLB)?
3. Какое практическое применение может иметь выделение виртуальных адресов, не обеспеченных физическими страницами памяти?
4. Что произойдет, если осуществить операцию чтения первого байта по виртуальному адресу, который не обеспечен физической страницей памяти?

Литература

1. *Таненбаум Э., Бос Х.* Современные операционные системы / пер. с англ. А. Леонтьевой, М. Малышевой, Н. Вильчинского. 4-е изд. СПб.: Питер, 2020. 1119 с. (Классика computer science).
2. *Олифер В.Г., Олифер Н.А.* Сетевые операционные системы: учебник. СПб.: Питер, 2001. 538 с.
3. *Крищенко В.А., Рязанова Н.Ю.* Основы программирования в ядре операционной системы GNU/LINUX : учеб. пособие. М.: Изд-во МГТУ им. Н.Э. Баумана, 2010. 34 с.
4. *Таненбаум Э., Остин Т.* Архитектура компьютера / пер. с англ. Е. Матвеева. 6-е изд. СПб.: Питер, 2022. 811 с. (Классика computer science).
5. *Домрачева А.Б.* Методические указания к выполнению лабораторных работ по курсу «Моделирование». М.: Изд-во МГТУ им. Н.Э. Баумана, 2016. 56 с.

Форма отчета о выполнении лабораторной работы по курсу «Операционные системы»



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Московский государственный технический
университет имени Н.Э. Баумана (национальный исследовательский
университет)» (МГТУ им. Н.Э. Баумана)

Факультет «Информатика и системы управления»
Кафедра «Теоретическая информатика и компьютерные технологии»

Отчет о лабораторной работе № [номер работы]
по курсу «Операционные системы»
[название работы]

Студент: [ФИО студента]
Группа: [индекс группы]
Преподаватель: [ФИО преподавателя]

Москва, [год выполнения работы]

Содержание

[Содержание]

Постановка задачи

[Что требуется сделать в лабораторной работе]

Практическая реализация

[Каким образом была проведена реализация. Привести важные части исходного кода; указывать весь исходный код обычно не требуется, в исключительных случаях это можно сделать в приложении]

Результаты

[Что именно было реализовано. Продемонстрировать результаты работы, используя снимки экрана, распечатки отладочного журнала и пр.]

Выводы

[Какие выводы были извлечены из процесса выполнения работы]

Пример: В ходе выполнения были изучены способы проектирования и реализации простейшего драйвера в операционных системах ReactOS и NetBSD. В операционной системе ReactOS это было проще в ... , но сопряжено с трудностями при В операционной системе NetBSD основной сложностью стало ... , но при этом ... заняло очень мало времени.

Список литературы

[Здесь размещается нумерованный список всех источников, из которых была заимствована информация для написания отчета. В тексте отчета обязательно должны быть даны ссылки в квадратных скобках после заимствованных фрагментов]

Содержание

Предисловие.....	3
Краткие теоретические сведения.....	5
Требования к выполнению лабораторных работ	14
Практическая часть	16
Работа № 1. ReactOS и NetBSD. Среда сборки, установка и тестирование в виртуальной машине.....	16
Работа № 2. Загружаемый модуль ядра (драйвер)	19
Работа № 3. Список процессов в системе.....	22
Работа № 4. Операции с виртуальной памятью	27
Литература.....	31
Приложение. Форма отчета о выполнении лабораторной работы по курсу «Операционные системы».....	32

Учебное издание

Брагин Алексей Владимирович
Иванов Игорь Потапович

Операционные системы

Редактор *Е.К. Кошелева*
Художник *Я.М. Асинкритова*
Корректор *А.Э. Анисимова*
Компьютерная графика *Т.К. Сегеды*
Компьютерная верстка *Е.В. Ляшкевич*

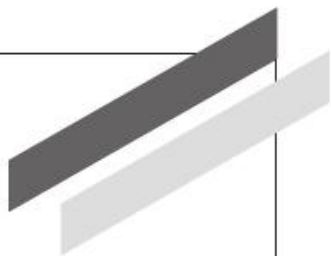
Оригинал-макет подготовлен
в Издательстве МГТУ им. Н.Э. Баумана.

В оформлении использованы шрифты
Студии Артемия Лебедева.

Подписано в печать 03.05.2024. Формат 60×90/16.
Усл. печ. л. 2,25. Тираж 69 экз. Изд. № 1410-2023.

Издательство МГТУ им. Н.Э. Баумана.
105005, г. Москва, улица 2-я Бауманская, д. 5, стр. 1.
press@bmstu.ru
<https://press.bmstu.ru>

Отпечатано в типографии МГТУ им. Н.Э. Баумана.
105005, г. Москва, улица 2-я Бауманская, д. 5, стр. 1.



ЭЛЕКТРОННО-БИБЛИОТЕЧНАЯ СИСТЕМА ИЗДАТЕЛЬСТВА МГТУ им. Н.Э. БАУМАНА

Сайт Издательства МГТУ им. Н.Э. Баумана (<https://press.bmstu.ru>) обеспечивает оперативный доступ студентов к учебной литературе в электронной форме. На сайте в настоящее время размещаются все издания, выпускаемые в Издательстве МГТУ им. Н.Э. Баумана.

Студентам:

- полнотекстовый поиск требуемых материалов как по каталогу, так и внутри самого документа
- рубрикаторы по областям знаний, ключевым словам, фамилиям авторов
- доступ к размещенным в каталоге изданиям путем просмотра на стационарном, мобильном или планшетном компьютере и смартфоне

Авторам:

- возможность оперативно вносить изменения и дополнения в текст учебных изданий
- использование в учебном процессе материалов сайта вне зависимости от выхода их печатной версии
- подготовка к публикации и печати монографий для повышения индекса научного цитирования автора и рейтинга Университета



press.bmstu.ru

